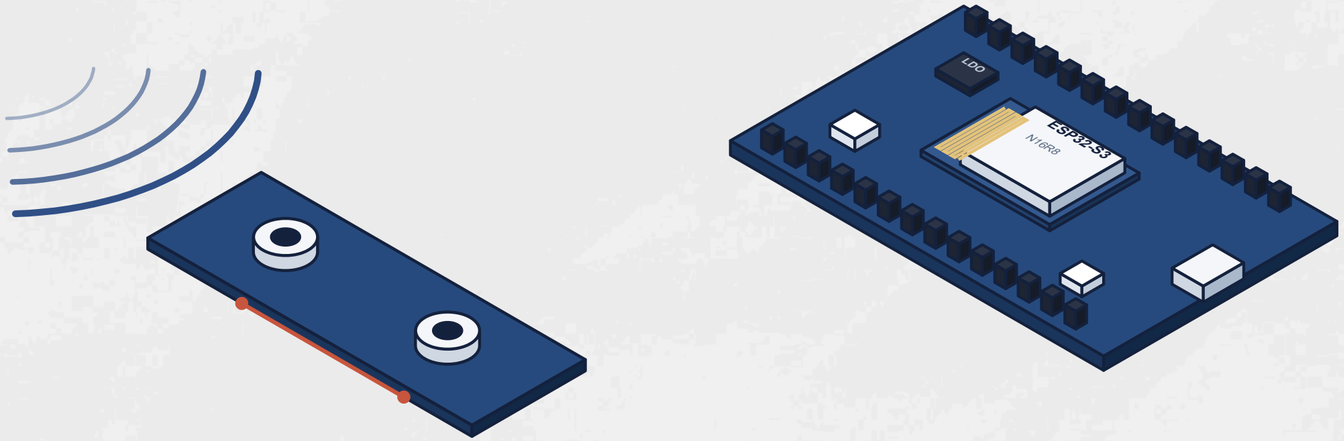


I. Beamforming de array endfire de micrófonos MEMS

II. Detección de palabra de activación en microcontrolador

Para la implementación en hardware de un asistente de IA para odontólogos



Dante Barili
Juan Förster
Mauricio Garavani
Bautista Mijalovich

Noviembre 2026

Órbita en el consultorio

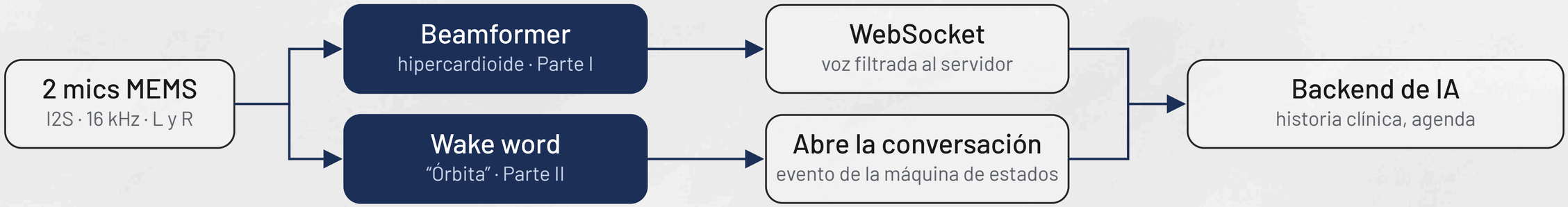
CONTEXTO

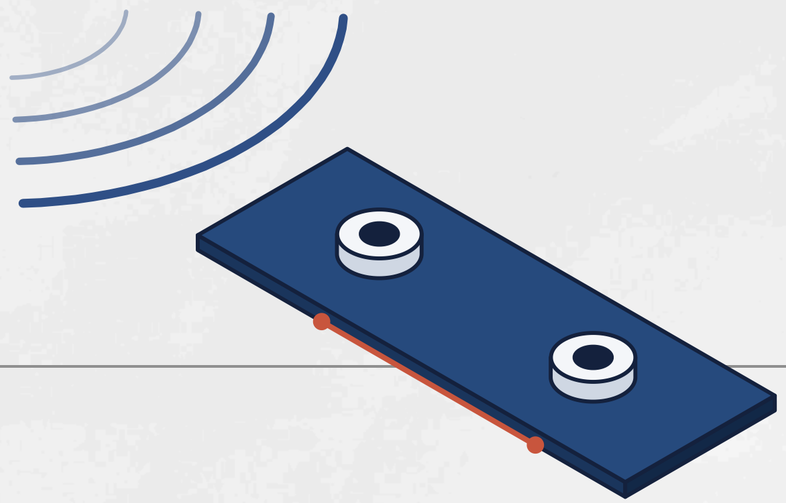


Arrastrá para girarlo

Dispositivo de escritorio sin pantalla: escucha la consulta, reconoce su nombre y manda el audio a una IA que completa la historia clínica.

Este trabajo cubre los dos bloques de **señales** que corren en el chip.





01

Beamforming

Cómo dos micrófonos separados 21,4 mm pueden preferir la voz que llega de frente: geometría, algoritmo, la demostración del hipercardioide y su ecualización.



Objetivo: directividad en un array chico

Captar la voz del odontólogo, que llega de frente, y no el ruido de la sala, que llega de todos lados.

Factor de directividad: lo que capta de frente contra el promedio de todas las direcciones.

$$\mathrm{DF} = \frac{|H(0)|^2}{\frac{1}{4\pi}\oint |H(\theta,\varphi)|^2\, d\Omega} = \frac{|H(0)|^2}{\frac{1}{2}\int_0^{\pi} |H(\theta)|^2 \sin\theta\, d\theta}$$

Los mics están mucho más juntos que la longitud de onda $\lambda = c/f$ (lo que avanza el sonido en un período): les llega casi igual.

$$\lambda_{500\,\mathrm{Hz}} = 69\ \mathrm{cm}, \quad \lambda_{3\,\mathrm{kHz}} = 11\ \mathrm{cm} \gg d = 21{,}4\ \mathrm{mm} \Rightarrow \omega\tau_0 \ll 1$$

6 dB

lo máximo que se puede lograr con 2 micrófonos: el **hipercardioide**

0 dB	−12 dB	−6 dB
voz de frente	costados	ruido de sala

Delay-and-sum contra delay-and-subtract

BEAMFORMING

En los dos casos al mic 2 se le agrega un retardo T. El retardo total entre las dos señales es $\Delta = T + \tau_0 \cos\theta$, con $\tau_0 = d/c$.

Suma (delay-and-sum)

$$y(t) = \tfrac{1}{2}\left[x_1(t) + x_2(t - T)\right]$$

$$\boxed{|H_+| = \left|\cos\tfrac{\omega\Delta}{2}\right|}$$

Con mics tan juntos $\omega\Delta \ll 1$ y el coseno vale ≈ 1 para todo θ: **la suma no distingue direcciones**.

Resta (delay-and-subtract)

$$y(t) = x_1(t) - x_2(t - T)$$

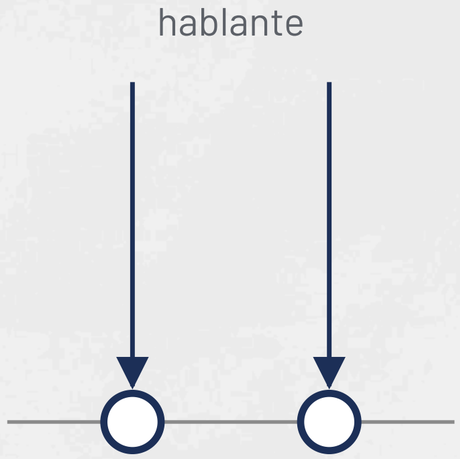
$$\boxed{|H_-| = 2\left|\sin\tfrac{\omega\Delta}{2}\right|} \qquad \Delta = 0 \iff \cos\theta_{\text{nulo}} = -\tfrac{T}{\tau_0}$$

Lo común se cancela: $|H_-| \approx \omega|\Delta|$, que **depende de θ**.

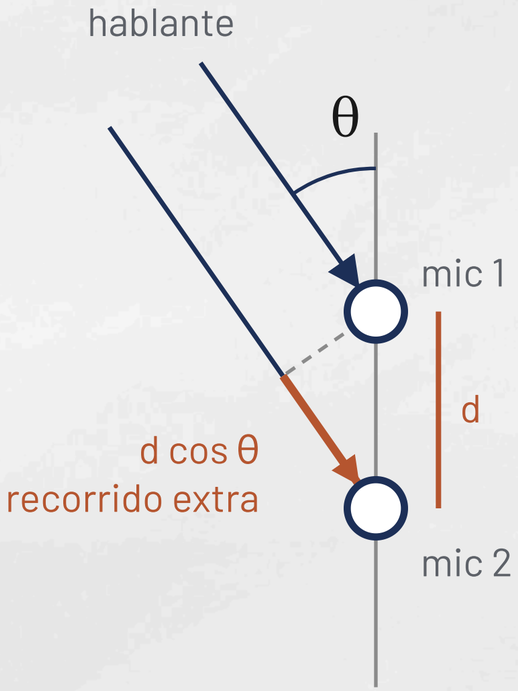
GANANCIA CONTRA RUIDO DIFUSO, D = 21 MM (SIMULACIÓN)	500 HZ	1 KHZ	4 KHZ
Delay-and-sum	0,0 dB	0,1 dB	0,9 dB
Cardioide (resta, T = τ_0)	4,7 dB	4,7 dB	3,0 dB
Hipercardioide (elegido)	**6,0 dB**	**6,0 dB**	**5,1 dB**

Geometría: broadside contra endfire

Broadside



Endfire



el eje apunta al hablante

Al segundo mic el sonido le llega más tarde:

$$\Delta x = d\cos\theta \;\Rightarrow\; \Delta t = \tau_0 \cos\theta$$

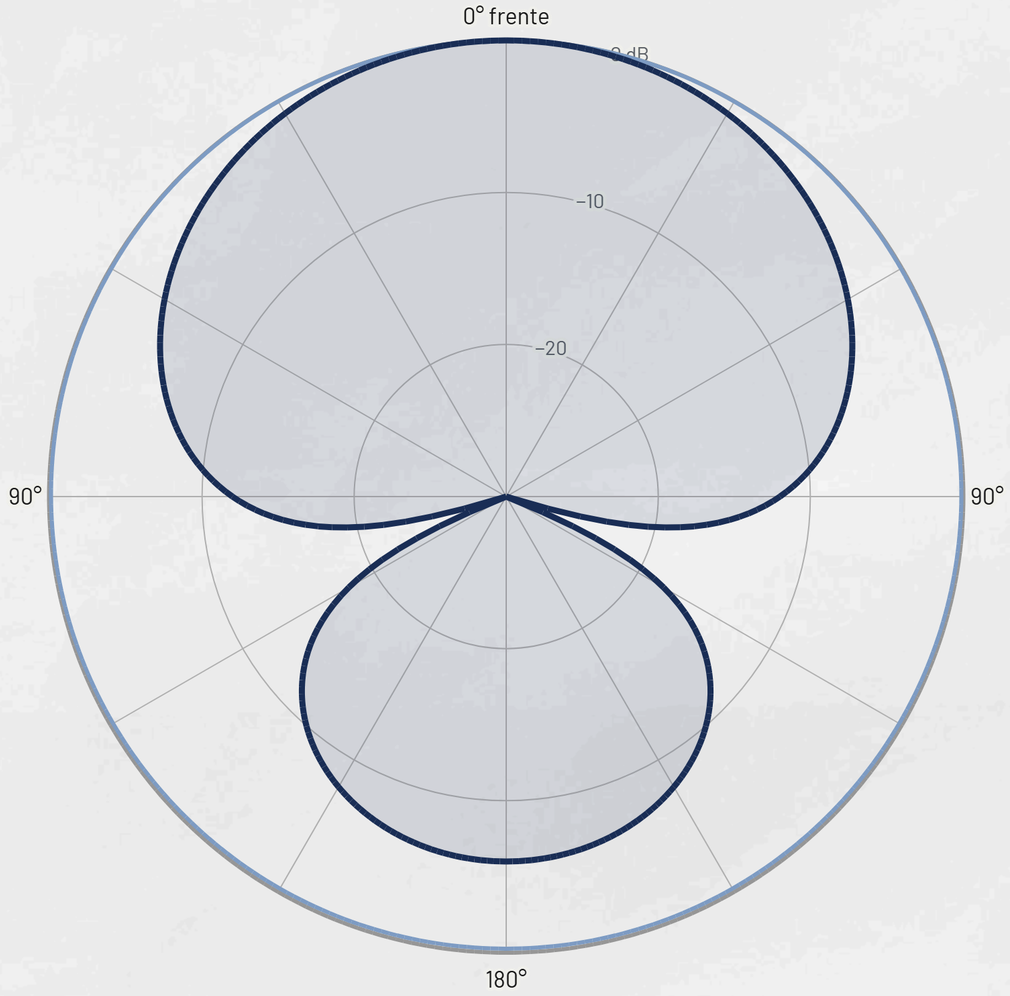
Con $\tau_0 = d/c$, la separación d da un retardo de **exactamente una muestra**: un z^{-1}, sin interpolar.

$$d = \frac{c}{f_s} = \frac{343}{16\,000} = 21{,}4\ \text{mm}$$

- **Broadside:** llega a los dos a la vez; la resta lo cancela.
- **Endfire:** $+\tau_0$ de frente, $-\tau_0$ de atrás; eso usa la resta.

Suma contra resta, en vivo

BEAMFORMING · INTERACTIVO



Frecuencia 630 Hz

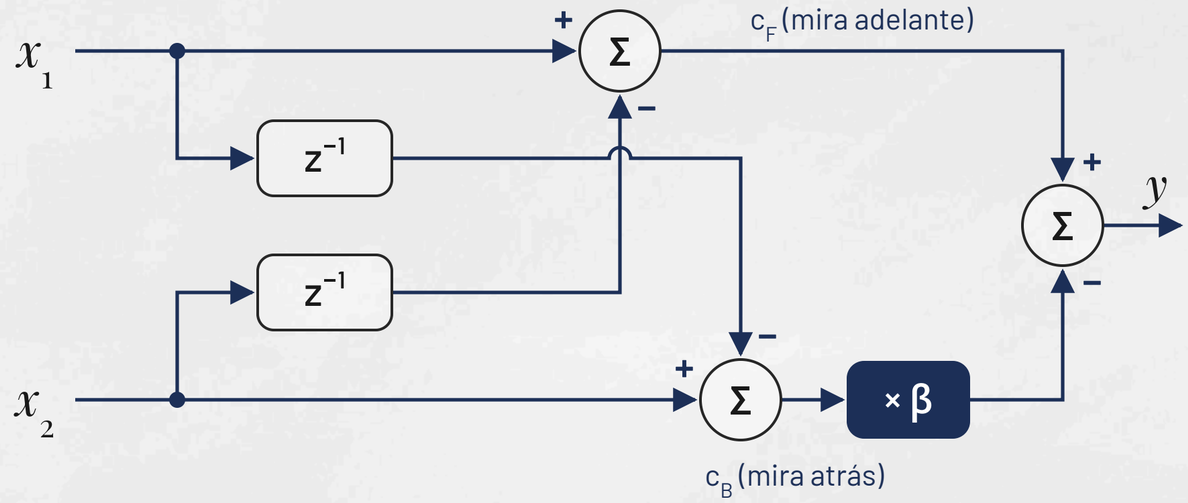
Separación d 21,5 mm

- [x] Delay-and-sum broadside
- [x] Delay-and-sum endfire
- [ ] Cardioide (resta)
- [x] Hipercardioide (resta)

ALGORITMO	DF	A 90°
Delay-and-sum broadside	0,0 dB	0,1 dB
Delay-and-sum endfire	0,1 dB	−0,1 dB
Cardioide	4,7 dB	−6,0 dB
Hipercardioide	**6,0 dB**	**−12,0 dB**

Curvas normalizadas a su frente; radio de 0 a −30 dB.

Implementación con dos cardioides



$$c_F[n] = x_1[n] - x_2[n-1]$$
$$c_B[n] = x_2[n] - x_1[n-1]$$
$$y[n] = c_F[n] - \beta\, c_B[n]$$

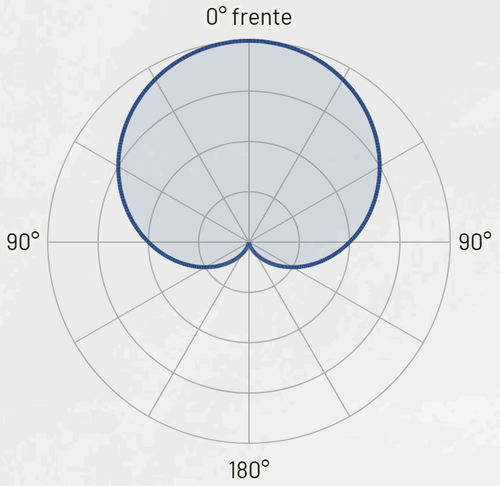
1. En graves ($\sin x \approx x$) los cardioides son
$$c_F \propto \tfrac{1}{2}(1+\cos\theta)$$
$$c_B \propto \tfrac{1}{2}(1-\cos\theta)$$

2. La resta es una mezcla de omni y figura-8:
$$y \propto B(\theta) = \alpha + (1-\alpha)\cos\theta$$
$$\alpha = \tfrac{1-\beta}{2}$$

3. Con $\beta = \frac{1}{2}$ sale el hipercardioide:
$$B(\theta) = \tfrac{1}{4} + \tfrac{3}{4}\cos\theta, \quad \theta_{\text{nulo}} = 109{,}5^\circ$$

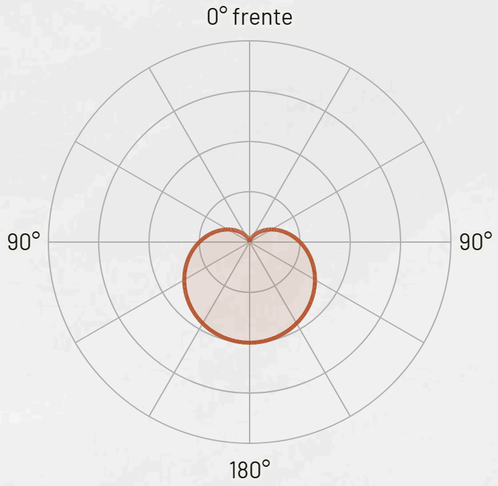
De dos cardioides al hipercardioide

BEAMFORMING · INTERACTIVO



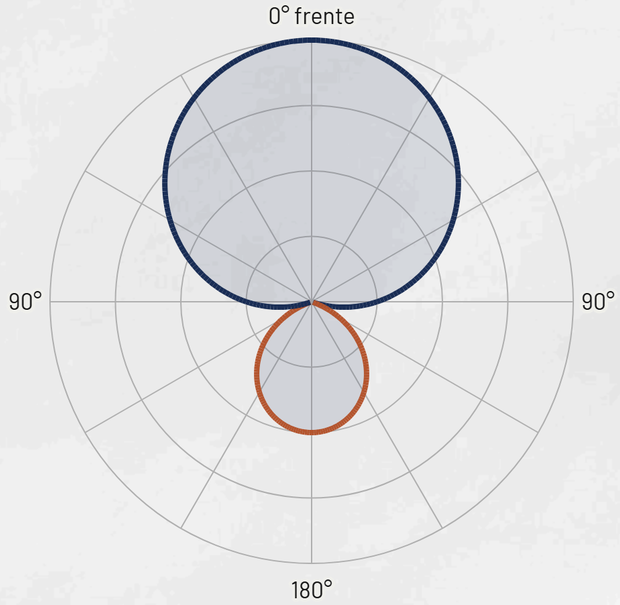
c_F

—



$0{,}50 \cdot c_B$

=



$y \cdot hipercardioide$

Peso β `0,50`

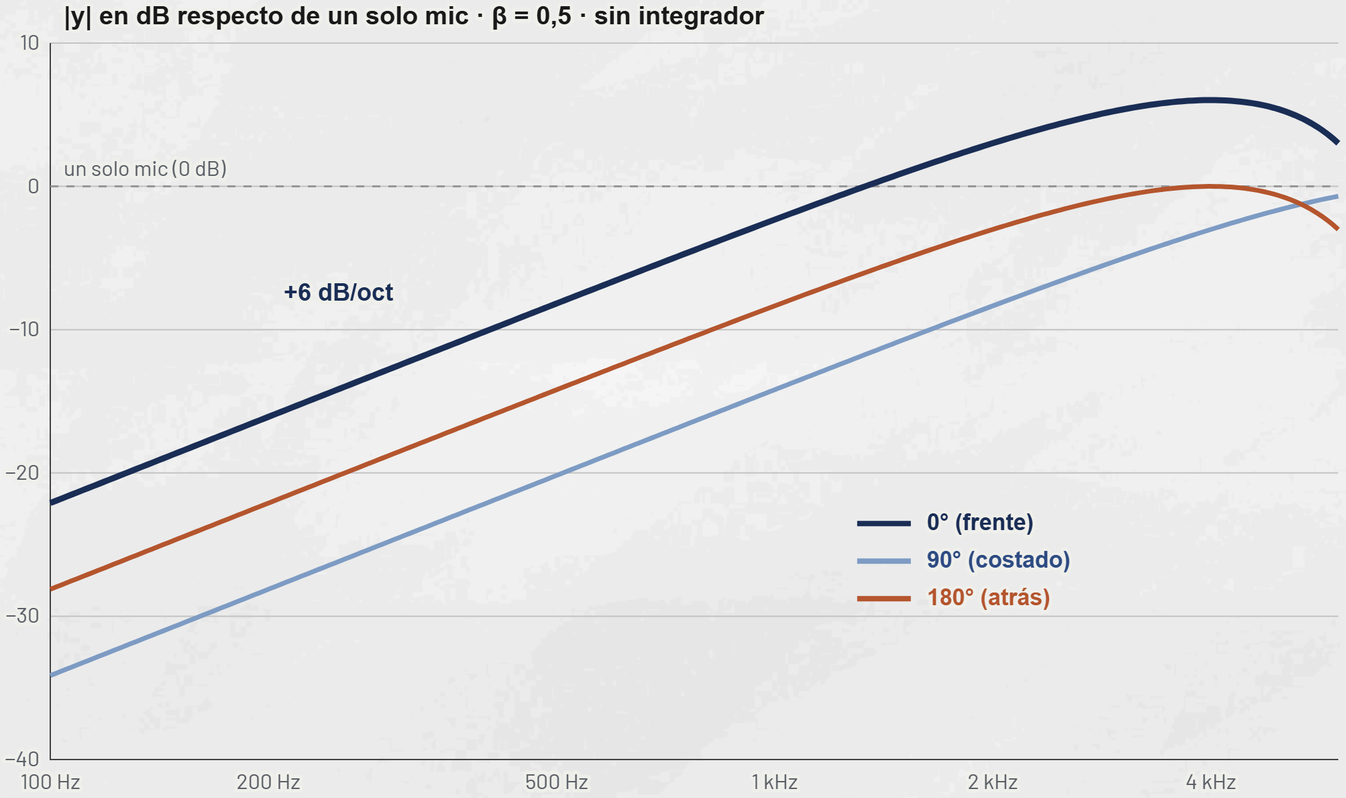
Modelo

Baja frecuencia (sin x ≈ x)

Costado (90°)	−12,0 dB
Atrás (180°)	−6,0 dB
Nulo	110°
DF	6,0 dB

Punteado: lóbulo con polaridad invertida. Pasá el mouse sobre y para leer cada ángulo.

Respuesta en frecuencia, sin corregir



En graves, restar es derivar: lo común se cancela y queda la pendiente.

$$s(t) - s(t-\Delta) \approx \Delta\, s'(t) \;\Rightarrow\; \times j\omega$$

$$|y| \approx 2\,\omega\tau_0\, B(\theta)$$

$$B(\theta) = \tfrac{1}{4} + \tfrac{3}{4}\cos\theta$$

- **+6 dB/oct:** el nivel sube con ω.
- A 200 Hz la voz de frente queda **−16,1 dB** bajo un solo mic.

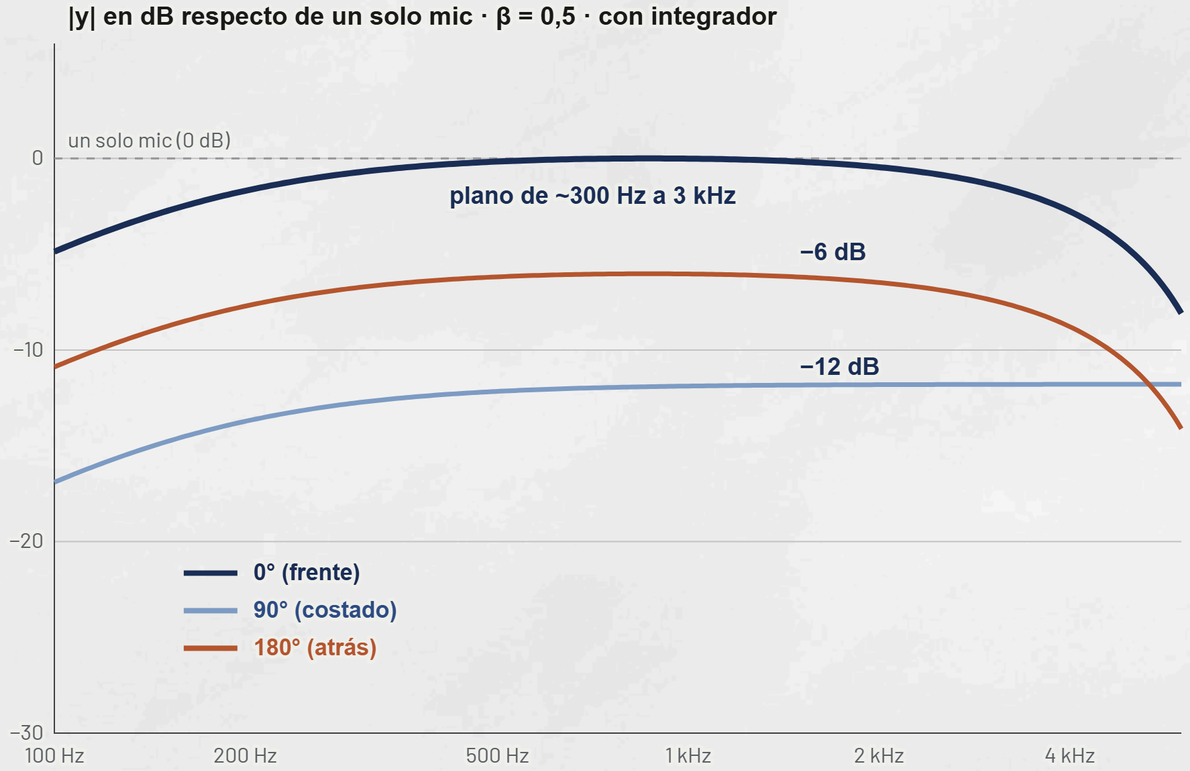
Modelo exacto, d = 21,4 mm. Pasando ~3 kHz la recta deja de valer (sin x ≠ x).

El ecualizador

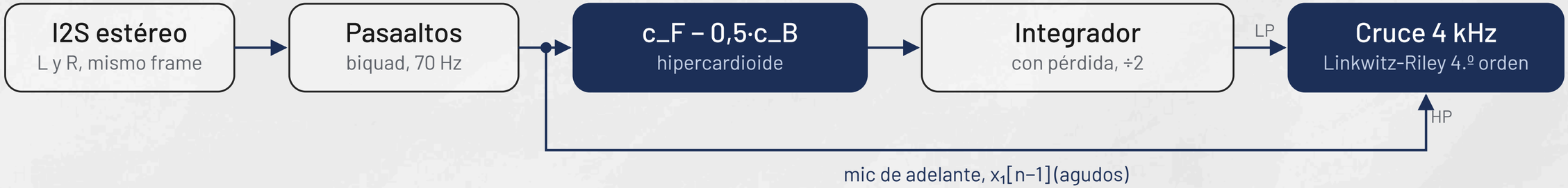
1. De frente $x_2[n] = x_1[n-1]$, así que $c_B = 0$ y
$$H_{\text{frente}}(z) = 1 - z^{-2} = (1 - z^{-1})(1 + z^{-1})$$
$1 - z^{-1}$ es un derivador: +6 dB/oct.

2. Se cancela con su inversa, el integrador:
$$(1 - z^{-2}) \cdot \frac{1}{1 - z^{-1}} = 1 + z^{-1}$$
$$\left|\tfrac{1}{2}(1 + z^{-1})\right| = \cos \tfrac{\omega}{2f_s} \approx 1$$

3. Con pérdida, para no saturar con la continua:
$$v[n] = a\, v[n-1] + y[n]$$
$$f_c = 150\ \text{Hz} \ \Rightarrow\ a = e^{-2\pi f_c / f_s} \approx 0{,}943$$



Implementación en el ESP32-S3



Por qué se corta a 4 kHz

Arriba de ~4 kHz el frente del hipercardioide cae (–3 dB a 4 kHz, –16 dB a 7 kHz) y se pierden consonantes. Arriba del cruce va el mic de adelante solo:

$$\text{salida} = \text{LP}(y) + \text{HP}\big(x_1[n-1]\big)$$

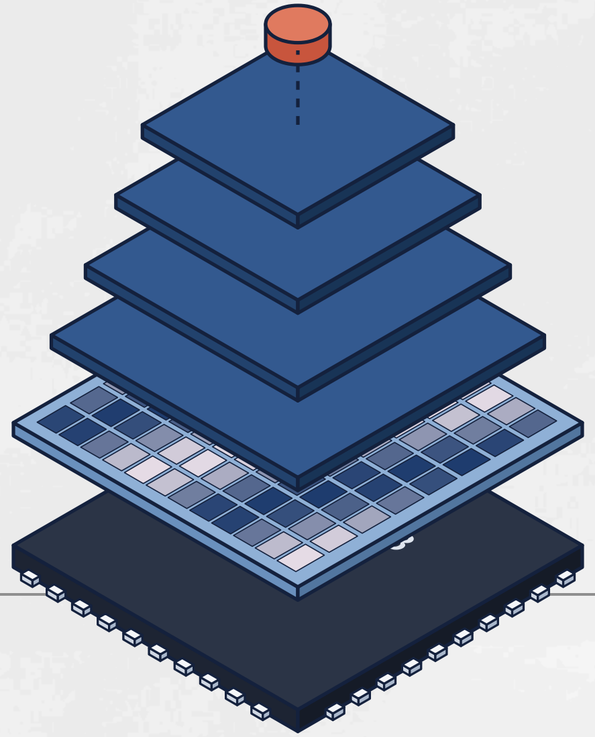
$x_1[n-1]$: una muestra atrás para quedar en fase con el hipercardioide, que de frente vale $\approx \frac{1}{2}(x_1[n] + x_1[n-1])$.

Qué cuesta

LP y HP son dos biquads Butterworth en cascada cada uno: sumados dan módulo plano.

Arriba de 4 kHz la directividad es la de un mic solo (0 dB). Ahí la voz tiene poca energía.

Simulado: frente dentro de –2 dB de 200 Hz a 7 kHz. **Pasaaltos** de 70 Hz antes de restar: el integrador sube los graves hasta 17,5 veces.



02

Wake word

Detectar “Órbita” en el microcontrolador: el chip, el framework, los datos, el espectrograma que ve la red, la red y cómo decide cada 30 ms.



Objetivo: escuchar una palabra, en el chip

El dispositivo escucha todo el tiempo, pero el audio sale a la red recién cuando alguien dice “Órbita”.

Una función aprendida, evaluada cada 30 ms, más un comparador:

$$f_{\mathbf{w}} : \underbrace{X \in \mathbb{R}^{\sim 150 \times 40}}_{\text{espectrograma de } \sim 1{,}5 \text{ s}} \longrightarrow p \in (0,1)$$

$$\text{detección} \iff \bar{p}[n] > p_{\text{umbral}}$$

Privacidad

en un consultorio no se transmite nada hasta la palabra

Latencia

la decisión no depende de la red

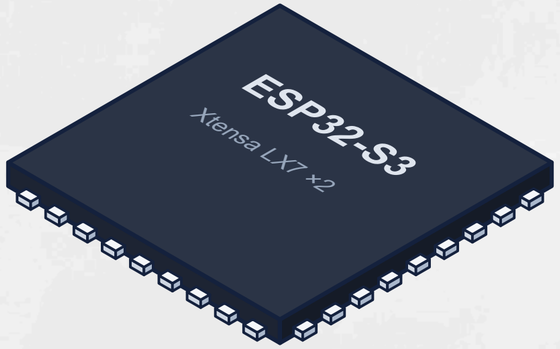
Costo

sin streaming continuo al servidor

Límite

tiene que entrar en KB de RAM y en una fracción de un núcleo

¿Por qué ESP32-S3?



DEL DATASHEET (V2.2, "FEATURES")	PARA QUÉ SIRVE ACÁ
2 núcleos Xtensa LX7, hasta 240 MHz	captura y DSP en uno, red en el otro
Bus de 128 bits e instrucciones SIMD	**registros de 128 bits: 16 valores int8 por instrucción**
512 KB de SRAM + PSRAM octal (8 MB en el N16R8)	modelo y arena en interna, buffers de audio en PSRAM
Dos interfaces I2S	mics por una, amplificador por la otra
Wi-Fi 802.11 b/g/n y Bluetooth LE 5	WebSocket al servidor, en el mismo chip

×14

convolución 1×1 int8 con ESP-NN frente a C portable (README de ESP-NN)

2300 → 54 ms

detección de personas (int8) sin y con ESP-NN, a 240 MHz

Framework: microWakeWord + TFLite Micro

microWakeWord (TensorFlow, GPU)
entrena MixedNet, la pasa a streaming y la cuantiza

PC

orbita.tflite · 61 KB
grafo + pesos int8, embebido en la flash

chip

TensorFlow Lite Micro
intérprete en C++, arena fija, sin malloc

ESP-NN
kernels int8 en assembler con SIMD del S3

ESP32-S3
ESP-IDF · FreeRTOS · núcleo 1

1. **Mismo frontend en PC y chip.** Si el espectrograma difiere, el modelo falla sin avisar.
2. **int8.** 4 veces menos memoria y cuentas enteras, que acelera el SIMD.
3. **Streaming.** Cada llamada procesa solo 30 ms nuevos.
4. **Memoria fija.** Arena reservada al arrancar, sin malloc.

Datos de entrenamiento

WAKE WORD

CONJUNTO	QUÉ ES	CANTIDAD (V0)	% DEL LOTE	PENALIDAD × CLASE
Positivos	**“Órbita” sintético, ElevenLabs, voces rioplatenses**	**11.727 clips**	**12 %**	**2 × 2 = 4**
Confundibles	palabras parecidas a “órbita”	5.659 clips	12 %	5 × 40 = 200
Español leído	OpenSLR, Common Voice	22,3 h	8 %	2,5 × 40 = 100
Español conversación	PRESEEA, CORdeBA	34,2 h	23 %	3 × 40 = 120
Negativos de microWakeWord	habla en inglés, charla de cena, ruido y música	~900 h	46 %	40 a 120
Evaluación	entrevistas en español: eligen el mejor punto y miden	6,3 h + 5,3 h	0 %	no entrena

Aumentación: ruido (SNR –5 a 20 dB), eco de salas reales y volumen. Las voces reales quedan para el test.

Features: del audio al espectrograma



$$X_m[k] = \sum_{n=0}^{L-1} w[n]\, x[n + mH]\, e^{-j2\pi kn/N}$$

con $L = 480$ (30 ms), $H = 160$ (10 ms) y $N = 512$.

$$w[n] = \tfrac{1}{2}\left(1 - \cos\frac{2\pi n}{L-1}\right), \qquad \mathrm{mel}(f) = 1127 \ln\left(1 + \frac{f}{700}\right)$$

$$E_m[b] = \sqrt{\sum_k W_b[k]\, |X_m[k]|^2}, \qquad b = 1..40$$

Las ventanas se solapan dos tercios: cada instante entra en 3 columnas seguidas.

Reducción de ruido: sigue el piso de cada banda y lo resta:

$$N_m[b] = (1 - s)\, N_{m-1}[b] + s\, E_m[b]$$

$$E'_m[b] = \max\left(E_m[b] - N_m[b],\ 0{,}05\, E_m[b]\right)$$

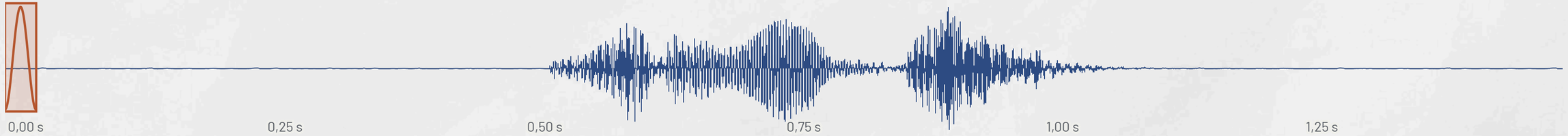
PCAN: un AGC por banda:

$$\mathrm{PCEN} = \left(\frac{E}{(\varepsilon + M)^{\alpha}} + \delta\right)^{r} - \delta^{r}$$

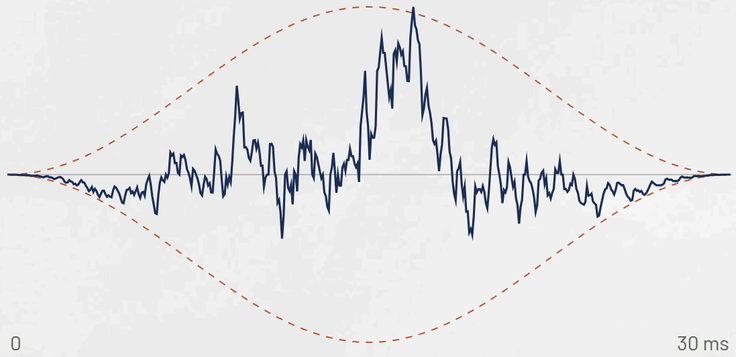
Forma de Wang et al. (2017); TFLM usa una versión entera.

La ventana que arma el espectrograma

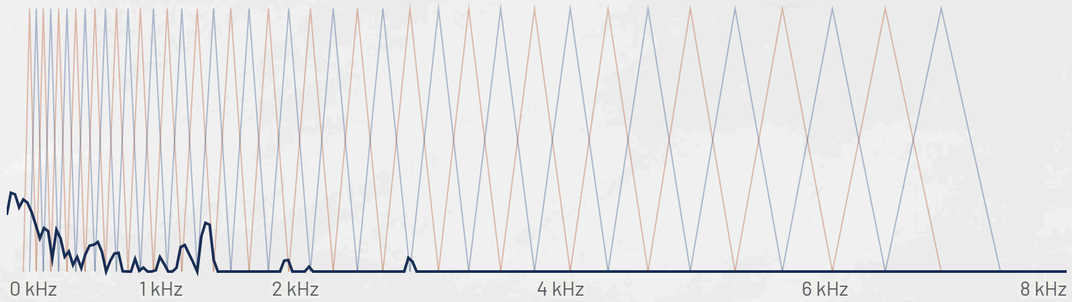
WAKE WORD · INTERACTIVO



Ventana × Hann (480 muestras)



|X[k]| en dB y los 40 filtros mel



columna



Columna

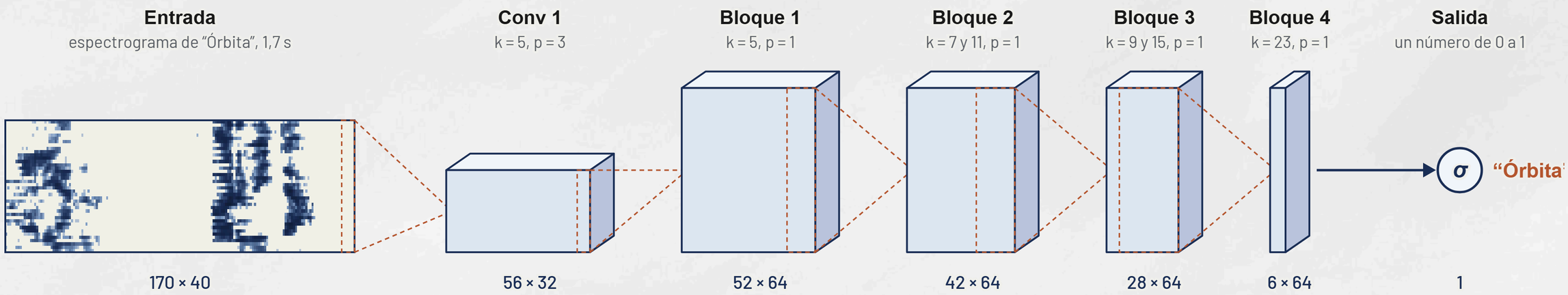
0 (0–30 ms)

Features del microfrontend (lo que ve la red): 40 bandas × una columna cada 10 ms



La red, etapa por etapa

k: columnas que mira cada filtro · **p**: paso, columnas que salta la ventana



tiempo → (ancho de cada bloque = pasos de tiempo; alto = canales)

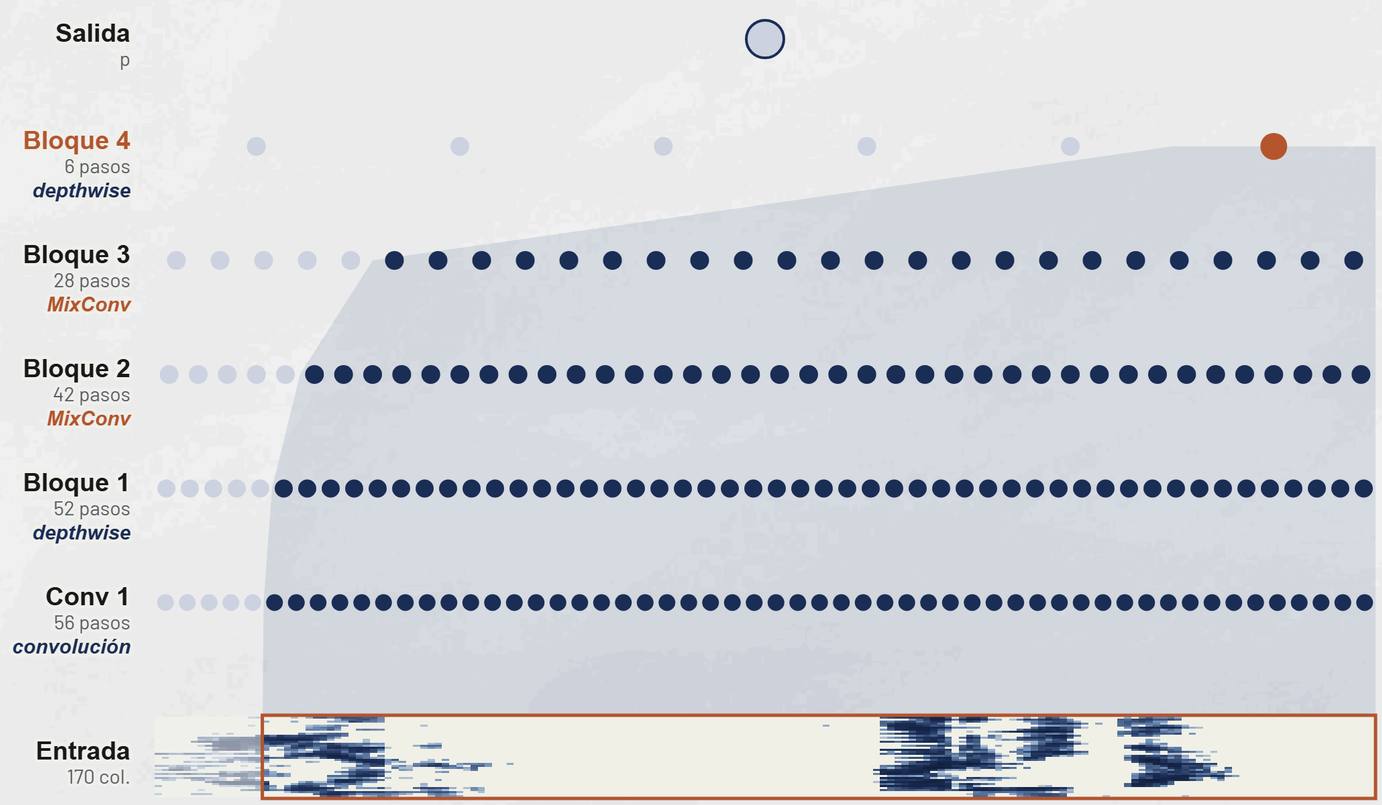
Paso 3: la ventana de Conv 1 salta de a 3 columnas, así que 170 → 56. Por eso cada inferencia procesa 30 ms nuevos.

La ventana punteada es lo que mira una salida: columnas enteras, porque las convoluciones son solo en el tiempo.

Cada bloque: un filtro en el tiempo para cada canal por separado (*depthwise*, un FIR por canal) y después una mezcla de canales (*pointwise*).

MixConv: la mitad de los canales con un filtro corto y la otra mitad con uno largo, para mirar dos escalas de tiempo a la vez.

La red: MixedNet



Pasá el mouse por una neurona para ver qué parte de la entrada mira.

¿Por qué MixedNet? Red convolucional diminuta (~24.000 pesos, 61 KB), hecha para un microcontrolador.

La entrada tiene 40 bandas. Después, cada **canal** es un patrón aprendido (32 o 64 por paso), ya no una frecuencia.

Depthwise: cada canal por separado. Una ventana de *k* pasos se desliza en el tiempo y los pondera con *k* pesos (el filtro).

MixConv: grupos de canales con ventanas de distinto largo (7 y 11 pasos): detalle corto y más contexto.

$$\underbrace{5 \cdot 64}_{\text{depthwise}} + \underbrace{64 \cdot 64}_{\text{pointwise}} = 4\,416 \ll \underbrace{5 \cdot 64 \cdot 64}_{20\,480}$$

$$y = \mathrm{ReLU}\Big(\sum_i w_i x_i + b\Big), \qquad p = \frac{1}{1 + e^{-z}}$$

Entrenamiento

Pérdida: entropía cruzada, pesada por conjunto:

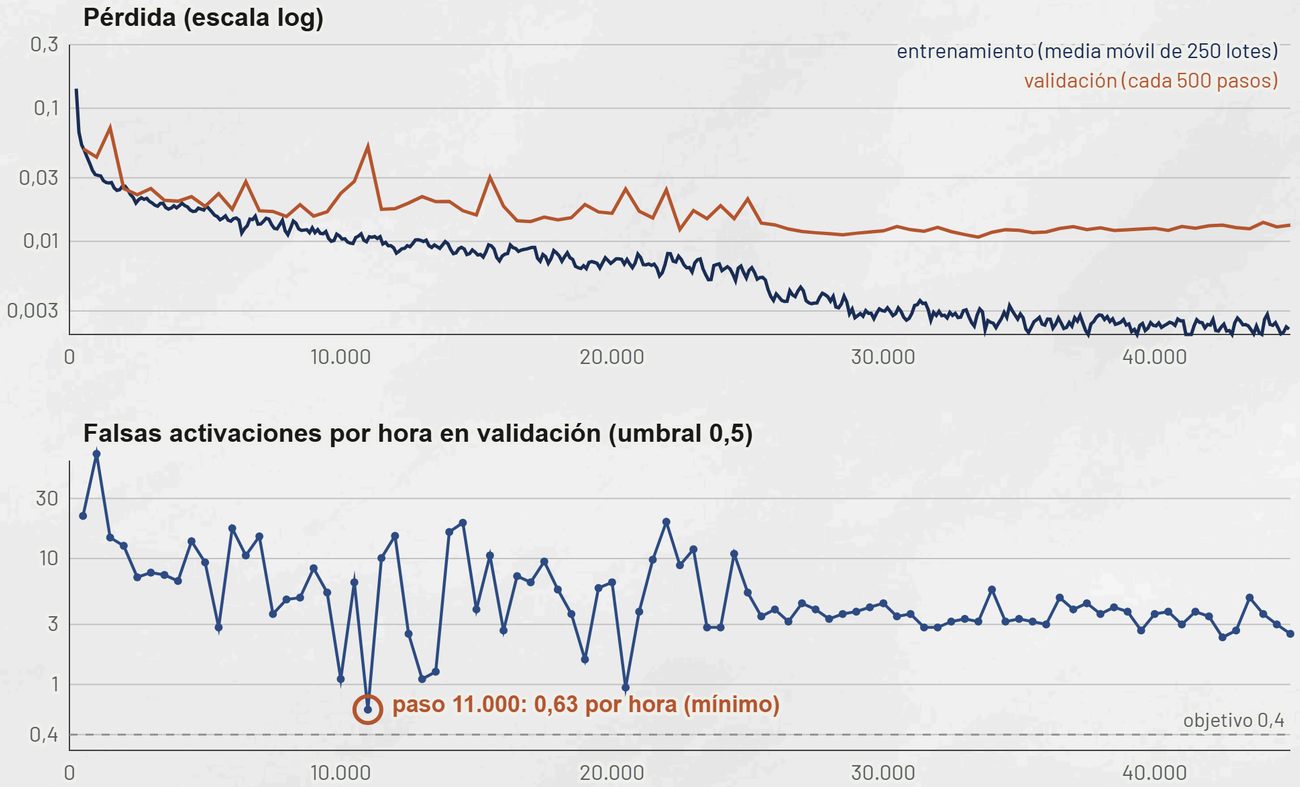
$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} c_i \left[y_i \ln p_i + (1 - y_i) \ln (1 - p_i) \right]$$

Descenso por gradiente (Adam):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \qquad \eta = 10^{-3} \rightarrow 10^{-4}$$

45.000 pasos en una A100 (~75 min). Mejor punto: **paso 11.000**.

La validación se estanca mientras el entrenamiento baja: empieza a memorizar.



Streaming y cuantización

Una red con memoria

Cada convolución guarda sus últimas entradas, como la línea de retardo de un FIR:

$$y[n] = \sum_{k=0}^{K-1} h[k]\, x[n-k]$$

$$\text{estado} = \{x[n-1], \ldots, x[n-K+1]\}$$

Cada llamada procesa solo **3 columnas nuevas** (30 ms).

Campo receptivo, en columnas de entrada:

$$5 + 3\,(4 + 10 + 14 + 22) = 155 \;\rightarrow\; 1{,}57\ \text{s}$$

De float32 a int8

Enteros de 8 bits con escala y cero, como un ADC:

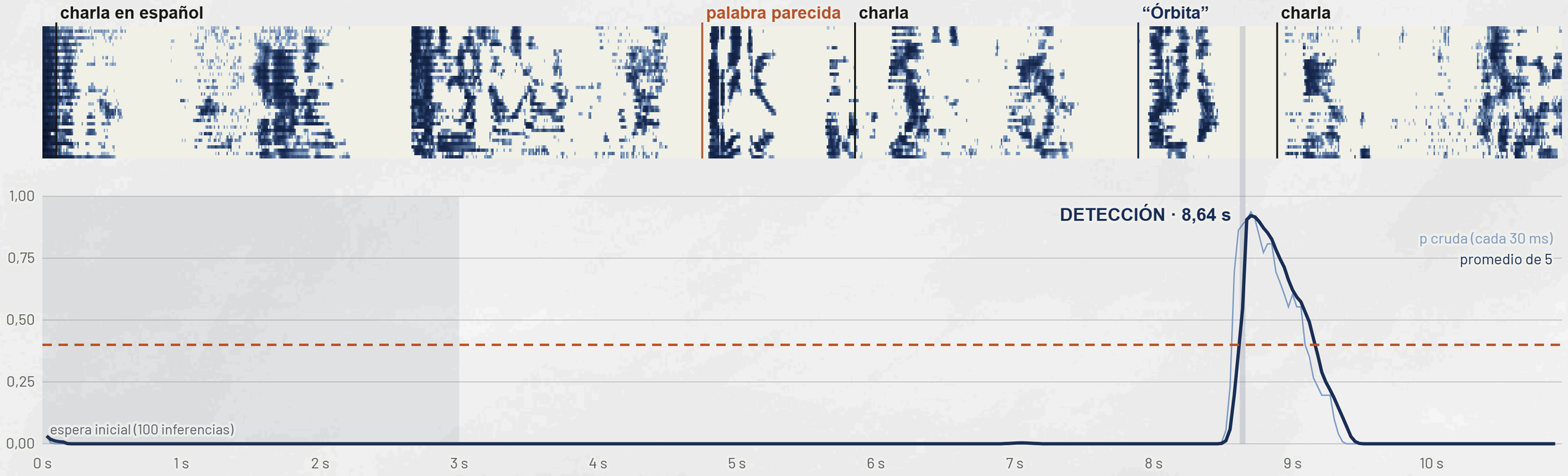
$$r = s\,(q - z), \qquad q = \text{clip}\Big(\text{round}\big(\tfrac{r}{s}\big) + z,\ -128,\ 127\Big)$$

Escalas medidas con 500 espectrogramas de entrenamiento.

	FORMA	TIPO
Entrada	[1, 3, 40]	int8
Salida	[1, 1]	uint8 → p = q/256

De la probabilidad a la detección

WAKE WORD · INTERACTIVO



Umbral 0,40

Promedio de N salidas 5 (150 ms)

$$\bar{p}[n] = \frac{1}{N}\sum_{k=0}^{N-1} p[n-k] \;>\; p_{\text{umbral}}$$

Detecciones 1

Máximo antes de “Órbita” 0,03

Audio real pasado por el **orbita.tflite** del firmware (int8, streaming).

Resultados en simulación

Entrenamiento v0 (paso 11.000, int8, umbral 0,4)

90 %	0	37 %
“Órbita” sintéticos detectados	falsas en 5,3 h de entrevistas	con 30 grabaciones reales

Hallazgo: la brecha entre voces sintéticas y reales. El v1 suma voces reales al entrenamiento.

Conclusiones y trabajo futuro

Conclusiones

- Con 2 mics tan juntos, **restar** rinde lo que sumar no: 6 dB de directividad debajo de 4 kHz; arriba, el mic de adelante solo.
- La resta sube +6 dB/oct; un integrador con pérdida la deja plana en la banda de voz.
- Una red int8 de ~24.000 pesos detecta el 90 % de los “Órbita” sintéticos, sin falsas en 5,3 h de entrevistas.

Trabajo futuro

- v1 del modelo con voces reales y falsas activaciones recolectadas.
- Medir en la placa: patrón del beamformer, detección por distancia y uso de CPU.
- Cancelación de eco para detectar mientras suena el parlante.

Referencias

1. G. W. Elko y A.-T. Nguyen Pong, “A simple adaptive first-order differential microphone”, IEEE WASPAA, 1995.
2. J. Benesty y J. Chen, *Study and Design of Differential Microphone Arrays*, Springer, 2012.
3. InvenSense, nota de aplicación “Microphone Array Beamforming” (broadside y endfire); ST, DT0117.
4. Y. Wang et al., “Trainable frontend for robust and far-field keyword spotting”, IEEE ICASSP, 2017 (PCEN).
5. M. Tan y Q. V. Le, “MixConv: Mixed depthwise convolutional kernels”, BMVC, 2019.
6. K. Ahrendt, *microWakeWord*, github.com/kahrendt/microWakeWord (commit 4665173).
7. TensorFlow Lite Micro y su microfrontend, github.com/tensorflow/tflite-micro.
8. Espressif, *ESP32-S3 Series Datasheet* v2.2; *ESP-NN*, github.com/espressif/esp-nn.



¿Preguntas?

Gracias. Código y simulaciones del proyecto Órbita: firmware ESP-IDF, simulaciones en Python y entrenamiento del modelo.